

作业：Linux 下 C 语言编译过程与静态库、动态库制作

- 学号：20241328
- 姓名：蔡贤俊
- 日期：2026-09-28
- 实验环境：openEuler 24.03 LTS (在 Windows 11 上通过 WSL2 安装运行)
- 工具链版本：gcc 12.3.1、make 4.4.1、git 2.43.0、binutils 2.41 (as、ar)

说明：本次作业全部在国产化操作系统 **openEuler 24.03 LTS** 上真机完成，文中每一张截图均为 openEuler 终端中实际编译、运行与测试的过程截图。

任务1：编写 hello.c、分步编译与 git 记录

1.1 编写代码

首先创建 `hello.c` 文件，内容如下：

```
#include <stdio.h>

int main() {
    printf("hello 20241328 蔡贤俊\n");
    return 0;
}
```

1.2 分步编译过程

openEuler 系统默认已经安装了 gcc，如果没有可以用 `sudo dnf install gcc` 安装。我们分四步完成编译：

(1) **编译预处理**：处理头文件、宏展开，生成预处理后的文件。

```
gcc -E hello.c -o hello.i
```

这一步得到的 `hello.i` 已经把 `stdio.h` 的内容插入进来了，所有宏都展开完成。

(2) **编译**：把 C 代码翻译成汇编代码，生成 `.s` 汇编文件。

```
gcc -S hello.i -o hello.s
```

(3) **汇编**：把汇编代码翻译成机器码，生成可重定位目标文件 `.o`。

```
as hello.s -o hello.o
```

(4) 链接：把目标文件和 C 标准库链接，生成可执行程序。

```
gcc hello.o -o hello
```

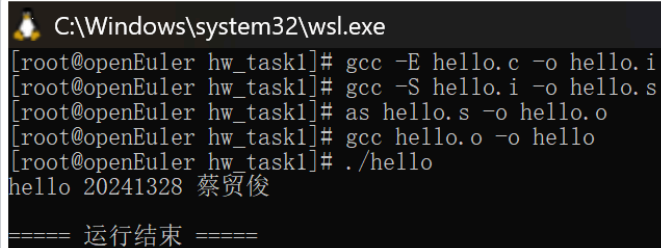
链接完成后我们就得到了可执行文件 `hello`，运行它：

```
./hello
```

运行结果：

```
hello 20241328 蔡贤俊
```

以上四个步骤（预处理 → 编译 → 汇编 → 链接）的实测过程如下图所示：



```
C:\Windows\system32\wsl.exe
[root@openEuler hw_task1]# gcc -E hello.c -o hello.i
[root@openEuler hw_task1]# gcc -S hello.i -o hello.s
[root@openEuler hw_task1]# as hello.s -o hello.o
[root@openEuler hw_task1]# gcc hello.o -o hello
[root@openEuler hw_task1]# ./hello
hello 20241328 蔡贤俊
===== 运行结束 =====
```

1.3 git 记录过程

首先初始化 git 仓库，然后一步步提交：

```
# 初始化git仓库
git init
# 添加hello.c到暂存区
git add hello.c
# 第一次提交：编写hello.c源代码
git commit -m "编写hello.c源代码"
# 添加预处理后的hello.i
```

```
git add hello.i
git commit -m "生成编译预处理文件hello.i"
# 添加编译后的汇编文件hello.s
git add hello.s
git commit -m "编译生成汇编文件hello.s"
# 添加汇编后的目标文件hello.o
git add hello.o
git commit -m "汇编生成目标文件hello.o"
# 添加链接后的可执行文件hello
git add hello
git commit -m "链接生成可执行文件hello"
```

这样就把整个过程都用 git 记录下来了。提交记录与各产物文件类型（`hello.o` 为可重定位目标文件，`hello` 为 ELF 可执行文件）如下图所示：



```
C:\Windows\system32\wsl.exe
[root@openEuler hw_task1]# ls -l
total 48
-rwxr-xr-x 1 root root 16240 Sep 28 16:17 hello
-rw-r--r-- 1 root root 90 Sep 28 16:15 hello.c
-rw-r--r-- 1 root root 18803 Sep 28 16:17 hello.i
-rw-r--r-- 1 root root 1520 Sep 28 16:17 hello.o
-rw-r--r-- 1 root root 497 Sep 28 16:17 hello.s
[root@openEuler hw_task1]# git log --oneline
22ac7d3 (HEAD -> master) 链接生成可执行文件hello
77e7551 汇编生成目标文件hello.o
c2fdb5a 编译生成汇编文件hello.s
4490ccc 生成编译预处理文件hello.i
ae61ac3 编写hello.c源代码
[root@openEuler hw_task1]# file hello.o hello
hello.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
hello: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux
===== 运行结束 =====
```

任务2：判断大端小端，实现两个判断函数

首先复习一下大端小端的概念：

- **大端模式**：数据的高字节保存在内存的低地址，低字节保存在高地址；
- **小端模式**：数据的低字节保存在内存的低地址，高字节保存在高地址。

我们用两种常用的方法实现，分别是指针法和共用体法。

2.1 代码编写

头文件放在 `include` 目录下，源代码放在 `src` 目录下。

`include/endian.h` 头文件内容：

```
#ifndef ENDIAN_H
#define ENDIAN_H

// 指针法判断大端小端
int is_little_endian_by_pointer();
// 共用体法判断大端小端
int is_little_endian_by_union();

#endif
```

`src/endian.c` 源代码内容：

```
#include "endian.h"

// 指针法：利用int型变量的低地址存放的字节判断
int is_little_endian_by_pointer() {
    unsigned int num = 0x12345678;
    unsigned char *p = (unsigned char *)&num;
    // 如果低地址存放的是0x78(低字节)，就是小端
    return (*p == 0x78);
}

// 共用体法：利用共用体所有成员共享起始地址的特性
int is_little_endian_by_union() {
    union {
        unsigned int num;
        unsigned char byte;
    } u;
    u.num = 0x12345678;
    // 起始地址的字节是低字节，就是小端
    return (u.byte == 0x78);
}
```

`src/main.c` 主函数内容：

```
#include <stdio.h>
#include "endian.h"

int main() {
    // 调用指针法判断
    if (is_little_endian_by_pointer()) {
```

```
        printf("指针法判断结果: 当前系统是小端模式\n");
    } else {
        printf("指针法判断结果: 当前系统是大端模式\n");
    }

    // 调用共用体法判断
    if (is_little_endian_by_union()) {
        printf("共用体法判断结果: 当前系统是小端模式\n");
    } else {
        printf("共用体法判断结果: 当前系统是大端模式\n");
    }

    return 0;
}
```

2.2 直接编译运行过程

不做库的话我们可以直接编译运行，命令如下：

```
gcc src/main.c src/endian.c -Iinclude -o endian_test
./endian_test
```

说明：因为 `endian.h` 放在 `include` 目录，编译时补充了 `-Iinclude` 指定头文件搜索路径。

运行结果：

```
指针法判断结果: 当前系统是小端模式
共用体法判断结果: 当前系统是小端模式
```

openEuler 运行在 x86 和 ARM 上都是小端，所以得到这个结果是正常的。

```
C:\Windows\system32\wsl.exe
[root@openEuler endian_hw]# gcc src/main.c src/endian.c -Iinclude -o endian_test
[root@openEuler endian_hw]# ./endian_test
指针法判断结果: 当前系统是小端模式
共用体法判断结果: 当前系统是小端模式
[root@openEuler endian_hw]# lscpu | grep 'Byte Order'
Byte Order:                                Little Endian

===== 运行结束 =====
```

任务3：制作静态库和共享库

先简单说明静态库和共享库的区别：

- **静态库**：在链接的时候会把库的代码复制到可执行程序中，程序运行不依赖外部库，体积更大；
- **共享库**：链接的时候只做标记，程序运行的时候才把库加载到内存，多个程序可以共享同一个库，体积小，更新方便。

3.1 制作静态库

静态库的后缀一般是 `.a`，制作步骤如下：

```
# 先把源代码编译成目标文件
gcc -c src/endian.c -Iinclude -o endian.o
# 用ar命令打包成静态库
ar rcs libendian.a endian.o
```

这样就得到了静态库 `libendian.a`，使用静态库编译运行：

```
gcc src/main.c -Iinclude -L. -lendian -o endian_static
./endian_static
```

运行结果和之前直接编译是一样的。用 `ldd` 可以看到可执行文件**没有**依赖 `libendian.so`，说明静态库代码已被复制进程序：

```
C:\Windows\system32\wsl.exe
[root@openEuler endian_hw]# gcc -c src/endian.c -Iinclude -o endian.o
[root@openEuler endian_hw]# ar rcs libendian.a endian.o
[root@openEuler endian_hw]# ls -l libendian.a
-rw-r--r-- 1 root root 1610 Sep 28 16:20 libendian.a
[root@openEuler endian_hw]# gcc src/main.c -Iinclude -L. -lendian -o endian_static
[root@openEuler endian_hw]# ./endian_static
指针法判断结果: 当前系统是小端模式
共用体法判断结果: 当前系统是小端模式
[root@openEuler endian_hw]# ldd endian_static
        linux-vdso.so.1 (0x0000757f96818000)
        libc.so.6 => /usr/lib64/libc.so.6 (0x0000757f96633000)
        /lib64/ld-linux-x86-64.so.2 (0x0000757f9681a000)

===== 运行结束 =====
```

3.2 制作共享库

共享库后缀是 `.so`，一步就可以生成：

```
gcc -shared -fPIC src/endian.c -Iinclude -o libendian.so
```

参数说明：`-shared` 表示生成共享库，`-fPIC` 表示生成位置无关代码，这是共享库必须的。

使用共享库编译运行：

```
gcc src/main.c -Iinclude -L. -lendian -o endian_shared
./endian_shared
```

如果运行的时候提示找不到 `libendian.so`，可以用这个命令临时解决：

```
export LD_LIBRARY_PATH=./:$LD_LIBRARY_PATH
./endian_shared
```

下图中先展示了不加 `LD_LIBRARY_PATH` 时"找不到共享库"的报错，再展示设置后正常运行，`ldd` 可见 `libendian.so => ./libendian.so`：

```

C:\Windows\system32\wsl.exe
[root@openEuler endian_hw]# gcc -shared -fPIC src/endian.c -Iinclude -o libendian.so
[root@openEuler endian_hw]# gcc src/main.c -Iinclude -L. -lendian -o endian_shared
[root@openEuler endian_hw]# ./endian_shared
./endian_shared: error while loading shared libraries: libendian.so: cannot open shared object file: No such file or directory
[root@openEuler endian_hw]# export LD_LIBRARY_PATH=.:$LD_LIBRARY_PATH
[root@openEuler endian_hw]# ./endian_shared
指针法判断结果: 当前系统是小端模式
共用体法判断结果: 当前系统是小端模式
[root@openEuler endian_hw]# ldd endian_shared
        linux-vdso.so.1 (0x000073a06ce2e000)
        libendian.so => ./libendian.so (0x000073a06ce21000)
        libc.so.6 => /usr/lib64/libc.so.6 (0x000073a06cc44000)
        /lib64/ld-linux-x86-64.so.2 (0x000073a06ce30000)

===== 运行结束 =====

```

任务4：按目录结构存放并编译使用库

按要求创建目录结构：

```
mkdir bin include lib src
```

然后把对应的文件放进去：头文件 `endian.h` 放到 `include`；源代码 `main.c` 和 `endian.c` 放到 `src`；生成的库放到 `lib`；生成的可执行文件放到 `bin`。最终结构就是：

```

├─ bin: 可执行文件
├─ include: 头文件
│   └─ endian.h
├─ lib: 动态库，静态库
│   ├── libendian.a
│   └─ libendian.so
└─ src: 源代码
    ├── main.c
    └─ endian.c

```

4.1 编译制作静态库过程

```

# 1. 编译src下的endian.c生成目标文件
gcc -c src/endian.c -Iinclude -o endian.o

```

```
# 2. 打包成静态库放到lib目录
ar rcs lib/libendian.a endian.o

# 3. 编译main.c, 链接静态库生成可执行文件放到bin
gcc src/main.c -Iinclude -Llib -lendian -o bin/endian_static

# 删除临时的目标文件
rm endian.o
```

4.2 编译制作共享库过程

```
# 一步生成共享库放到lib目录
gcc -shared -fPIC src/endian.c -Iinclude -o lib/libendian.so

# 编译main.c链接共享库生成可执行文件放到bin
gcc src/main.c -Iinclude -Llib -lendian -o bin/endian_shared
```

按目录结构编译制作静态库与共享库的过程：

```
C:\Windows\system32\wsl.exe
[root@openEuler endian_hw]# mkdir -p bin include lib src
[root@openEuler endian_hw]# gcc -c src/endian.c -Iinclude -o endian.o
[root@openEuler endian_hw]# ar rcs lib/libendian.a endian.o
[root@openEuler endian_hw]# gcc src/main.c -Iinclude -Llib -lendian -o bin/endian_static
[root@openEuler endian_hw]# gcc -shared -fPIC src/endian.c -Iinclude -o lib/libendian.so
[root@openEuler endian_hw]# gcc src/main.c -Iinclude -Llib -lendian -o bin/endian_shared
[root@openEuler endian_hw]# rm endian.o
[root@openEuler endian_hw]# ls bin lib
bin:
endian_shared  endian_static  Makefile

lib:
libendian.a  libendian.so  Makefile

===== 运行结束 =====
```

4.3 运行可执行文件

运行静态库版本：

```
./bin/endian_static
```

运行共享库版本，因为共享库在 `lib` 目录，需要告诉系统去哪里找：

```
export LD_LIBRARY_PATH=./lib:$LD_LIBRARY_PATH
./bin/endian_shared
```

两个运行结果都是一样的，都能正确输出大小端判断结果。运行结果与最终目录结构（tree）如下：

```
C:\Windows\system32\wsl.exe
[root@openEuler endian_hw]# ./bin/endian_static
指针法判断结果：当前系统是小端模式
共用体法判断结果：当前系统是小端模式
[root@openEuler endian_hw]# export LD_LIBRARY_PATH=./lib:$LD_LIBRARY_PATH
[root@openEuler endian_hw]# ./bin/endian_shared
指针法判断结果：当前系统是小端模式
共用体法判断结果：当前系统是小端模式
[root@openEuler endian_hw]# tree .
.
├── bin
│   ├── endian_shared
│   ├── endian_static
│   └── Makefile
├── include
│   └── endian.h
├── lib
│   ├── libendian.a
│   ├── libendian.so
│   └── Makefile
└── src
    ├── endian.c
    ├── endian.o
    ├── main.c
    ├── main.o
    └── Makefile

5 directories, 12 files

===== 运行结束 =====
```

任务5：补充 Makefile

按要求在各个目录写 Makefile，不用变量，只用简单依赖关系：`src/Makefile` 用来编译目标文件；`lib/Makefile` 用来生成静态库和动态库；`bin/Makefile` 用来生成最终可执行文件。

`src/Makefile` 内容：

```
endian.o: ../src/endian.c ../include/endian.h
    gcc -c -fPIC ../src/endian.c -I../include -o endian.o

main.o: ../src/main.c ../include/endian.h
    gcc -c ../src/main.c -I../include -o main.o
```

`lib/Makefile` 内容：

```
libendian.a: ../src/endian.o
    ar rcs libendian.a ../src/endian.o
```

```
libendian.so: ../src/endian.o
gcc -shared -fPIC -o libendian.so ../src/endian.o
```

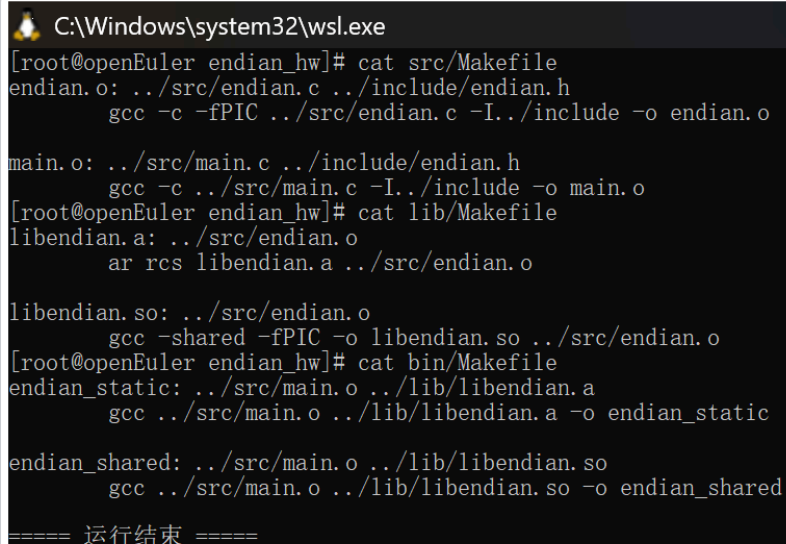
bin/Makefile 内容（只用简单依赖关系）：

```
endian_static: ../src/main.o ../lib/libendian.a
gcc ../src/main.o ../lib/libendian.a -o endian_static

endian_shared: ../src/main.o ../lib/libendian.so
gcc ../src/main.o ../lib/libendian.so -o endian_shared
```

说明：libendian.so 需要位置无关代码，因此 src/Makefile 中编译 endian.o 时补充了 -fPIC；另外 make 默认只构建第一个目标，所以 src 目录下需要额外执行 make main.o 生成 main.o。

三个 Makefile 的实际内容：



```
C:\Windows\system32\wsl.exe
[root@openEuler endian_hw]# cat src/Makefile
endian.o: ../src/endian.c ../include/endian.h
gcc -c -fPIC ../src/endian.c -I../include -o endian.o

main.o: ../src/main.c ../include/endian.h
gcc -c ../src/main.c -I../include -o main.o
[root@openEuler endian_hw]# cat lib/Makefile
libendian.a: ../src/endian.o
ar rcs libendian.a ../src/endian.o

libendian.so: ../src/endian.o
gcc -shared -fPIC -o libendian.so ../src/endian.o
[root@openEuler endian_hw]# cat bin/Makefile
endian_static: ../src/main.o ../lib/libendian.a
gcc ../src/main.o ../lib/libendian.a -o endian_static

endian_shared: ../src/main.o ../lib/libendian.so
gcc ../src/main.o ../lib/libendian.so -o endian_shared

===== 运行结束 =====
```

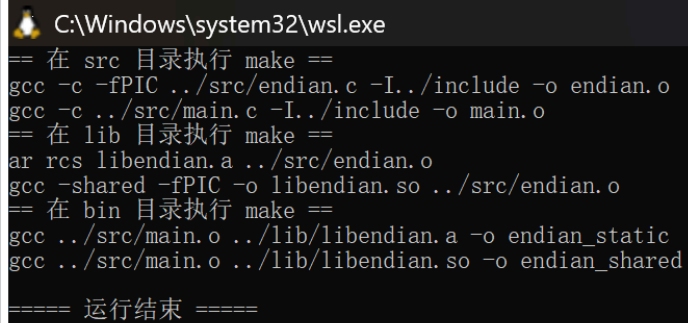
按顺序在各个目录执行 make：

```
# 在src目录编译目标文件
cd src
make
cd ..

# 在lib目录生成库
cd lib
```

```
make
cd ..
# 在bin目录生成可执行文件
cd bin
make
cd ..
```

使用 make 构建的过程：



```
C:\Windows\system32\wsl.exe
== 在 src 目录执行 make ==
gcc -c -fPIC ../src/endian.c -I../include -o endian.o
gcc -c ../src/main.c -I../include -o main.o
== 在 lib 目录执行 make ==
ar rcs libendian.a ../src/endian.o
gcc -shared -fPIC -o libendian.so ../src/endian.o
== 在 bin 目录执行 make ==
gcc ../src/main.o ../lib/libendian.a -o endian_static
gcc ../src/main.o ../lib/libendian.so -o endian_shared
===== 运行结束 =====
```

构建完成后运行，与之前步骤一样，均正确输出大小端判断结果：

```
C:\Windows\system32\wsl.exe
[root@openEuler endian_hw]# ./bin/endian_static
指针法判断结果: 当前系统是小端模式
共用体法判断结果: 当前系统是小端模式
[root@openEuler endian_hw]# cd bin && LD_LIBRARY_PATH=./lib ./endian_shared
指针法判断结果: 当前系统是小端模式
共用体法判断结果: 当前系统是小端模式
[root@openEuler endian_hw]# tree .
.
├── bin
│   ├── endian_shared
│   ├── endian_static
│   └── Makefile
├── include
│   └── endian.h
├── lib
│   ├── libendian.a
│   ├── libendian.so
│   └── Makefile
└── src
    ├── endian.c
    ├── endian.o
    ├── main.c
    ├── main.o
    └── Makefile

5 directories, 12 files

===== 运行结束 =====
```

附：总结

本次作业在 **openEuler 24.03 LTS** 上完整实践了 C 语言从源码到可执行文件的编译流程（预处理、编译、汇编、链接四步），掌握了：

1. 用 `git` 分步记录编译过程；
2. 用**指针法**和**共用体法**判断系统大端/小端（本机为小端）；
3. 静态库（`.a`）与共享库（`.so`）的制作与使用，以及两者的区别；
4. 规范的工程目录结构（`bin` / `include` / `lib` / `src`）；
5. 用 `Makefile` 简单依赖关系自动化构建。

这些都是在国产化系统上进行 C/C++ 开发最基础的编译与库知识，对后续开发很有帮助。